

Versión 5 — Tarea_Agente_Personal_RAG (Sesión 13, RAG real con Chroma)

Resumen

Es v1 (Sesión 12) con un cambio central: `buscar_en_documentos` deja de ser una búsqueda de texto literal y pasa a ser **búsqueda semántica real** sobre una base de datos vectorial (Chroma), con embeddings generados localmente vía Ollama. Se agregan además una tool de búsqueda web (Tavily) como respaldo, y salida estructurada (Pydantic) en vez de texto libre. A diferencia de v3/v4, no es una app web — sigue siendo un agente de consola de un solo archivo (más un módulo `rag.py` separado para el índice), porque el entregable de esta tarea es un script + un Google Doc, no una aplicación.

Dos archivos: `rag.py` (índice vectorial, sin dependencia de LangChain `create_agent`) y `agente_planificacion_rag.py` (el agente en sí, que importa `rag.py`).

`rag.py` — índice vectorial

Constantes y estado de módulo

`CARPETA_MATERIALES`, `CARPETA_INDICE` (`./chroma_index`, persistente en disco) y `NOMBRE_COLECCION` fijan dónde vive todo. `MODELO_EMBEDDINGS` / `OLLAMA_BASE_URL` se leen de variables de entorno con defaults (`nomic-embed-text` en `localhost:11434`), para no acoplar el código a una sola máquina. `_embeddings` / `_vector_store` son variables globales de módulo que cachean el cliente de embeddings y el vector store ya contruidos, para no recrearlos en cada llamada a `buscar()`.

`_obtener_embeddings()`

Lazy-init de `OllamaEmbeddings`: solo se instancia la primera vez que se necesita (evita abrir una conexión al servidor de Ollama si el índice ya existe en disco y no hace falta reindexar).

`_extraer_texto_archivo(ruta)`

Extrae texto plano según la extensión: `.txt/.md/.py` se leen directo, `.docx` usa `python-docx` (concatena el texto de cada párrafo), `.pdf` usa `pypdf` (concatena el texto extraído de cada página — importante: si el PDF no tiene capa de texto en una página, esa página simplemente no aporta nada, no falla).

`_cargar_documentos()`

Recorre `materiales/` recursivamente (`rglob("*")`), filtra por extensión admitida, extrae el texto de cada archivo, y lo divide en fragmentos con `RecursiveCharacterTextSplitter` (`chunk_size=800` , `chunk_overlap=120` — el solape evita perder contexto que quedaría cortado justo en el borde de dos fragmentos). Por cada fragmento crea un `Document` de `LangChain` con `metadata {"archivo": ..., "ruta_contexto": ...}` — `ruta_contexto` se calcula relativo a la carpeta *padre* de `materiales/` (no a `materiales/` misma), para que su valor coincida exactamente con el campo `ruta_contexto` de `tareas.json` (ej. `"materiales/TareaReact"`) y así poder filtrar la búsqueda por tarea más adelante. Cada fragmento recibe un id único determinístico (`archivo::índice_de_chunk`).

`construir_o_cargar_indice(forzar_reindexado=False)`

Si `forzar_reindexado=True` borra `chroma_index/` primero (reindexado completo desde cero). Si la carpeta del índice no existe todavía, indexa (`_cargar_documentos()` + `add_documents`); si ya existe y no se fuerza, simplemente abre el índice existente sin volver a leer/embeber los documentos (evita recalcular embeddings en cada arranque — son costosos). Se puede invocar manualmente por CLI: `python rag.py --reindexar` .

`buscar(consulta, k=4, ruta_contexto=None)`

La función que usa el resto del proyecto. Llama a `similarity_search` de `Chroma`, que compara el embedding de la consulta contra los embeddings indexados y devuelve los `k` más cercanos. Si se pasa `ruta_contexto` , se agrega como filtro exacto de metadata (`filter={"ruta_contexto": ...}`) para limitar la búsqueda a una carpeta/tarea específica.

`agente_planificacion_rag.py` — el agente

Qué se reutiliza de v1 sin cambios

`_cargar_tareas` , `_guardar_tareas` , `_buscar_tarea` , `_validar_ruta` , `_puntaje_urgencia` , y las `tools` `consultar_tareas` , `agregar_tarea` , `calcular_prioridad` , `generar_plan` , `actualizar_estado` , `guardar_plan_detallado` , `inspeccionar_carpeta` — mismo código, mismo

comportamiento. La gestión de tareas (`tareas.json`) no cambia con RAG; solo cambia cómo se consultan los *documentos*.

`buscar_en_documentos(consulta, tarea_id=None)` — la tool nueva

Reemplaza a la de v1 (que hacía `for linea in texto.splitlines(): if termino in linea`). Ahora:

si se da `tarea_id`, resuelve la tarea y usa su `ruta_contexto` como filtro; llama a `rag.buscar(consulta, k=4, ruta_contexto=...)`; devuelve los fragmentos encontrados citando el

archivo de origen (`[archivo] contenido`). La diferencia clave frente a v1: encuentra contenido

relevante aunque la pregunta no use las palabras exactas del documento (verificado en pruebas

reales — ver `ENTREGA_GOOGLE_DOC.md`).

`buscar_en_la_web(consulta)` — tool de respaldo

Usa `tavily-python` para buscar en la web cuando los materiales locales no alcanzan. Si no hay

`TAVILY_API_KEY` configurada, devuelve un mensaje explícito de "no disponible" en vez de fallar

con una excepción — el agente puede reportar esa limitación al usuario en vez de romperse o inventar una respuesta.

`RespuestaAgente (Pydantic)` — salida estructurada

En vez de que el agente devuelva texto libre, `create_agent(..., response_format=RespuestaAgente)`

lo obliga a llenar 4 campos: `resumen`, `acciones_recomendadas`, `fuentes` (qué documentos o resultados web citó) y `tools_usadas` (qué herramientas llamó realmente). Esto hace la respuesta

trazable: se puede verificar que el agente sí usó una tool antes de responder, en vez de confiar ciegamente en el texto.

`PROMPT_SISTEMA`

Mismo flujo que v1 (consultar → priorizar → planificar → buscar en materiales → actualizar estado), con dos agregados: cuándo usar `buscar_en_la_web` como respaldo (solo si

`buscar_en_documentos` no encontró nada útil), y una regla explícita agregada **después de un bug real encontrado en pruebas**: el modelo, en una primera prueba, respondió "dame un momento

mientras consulto tus materiales" como respuesta *final* sin haber llamado ninguna tool (fuentes / tools_usadas vacíos). La salida estructurada por sí sola no obliga a usar las tools antes de responder — se corrigió agregando al prompt una prohibición explícita de responder con mensajes de "espera" como respuesta final.

Arranque e inicialización

Al importar el módulo se llama `rag.construir_o_cargar_indice(forzar_reindexado="--reindexar" in sys.argv)` — así el índice está listo antes de crear el agente, y `python agente_planificacion_rag.py --reindexar` fuerza reconstruirlo sin tener que llamar `rag.py` por separado.

`iniciar_chat()`

Mismo loop de consola que v1, con dos cambios: en vez de imprimir `resultado["messages"][-1]` como texto libre, lee `resultado["structured_response"]` (una instancia de `RespuestaAgente`) e imprime sus 4 campos por separado; y en vez de reconstruir el historial a mano, usa `resultado["messages"]` completo como el `historial` del siguiente turno (conserva el registro completo de llamadas a tools entre turnos, no solo el texto final).

Qué cambia respecto a v1 (resumen)

	v1	RAG (v5)
Búsqueda en documentos	Texto literal (<code>in</code>), sin tildes	Semántica (Chroma + embeddings Ollama)
Filtrado por tarea	Implícito (busca en la carpeta de la tarea)	Explícito por metadata (<code>ruta_contexto</code>)
Fuente adicional	Ninguna	Búsqueda web (Tavily) como respaldo
Formato de respuesta	Texto libre	Estructurado (Pydantic: resumen/acciones/fuentes/tools)
Tools totales	8	9 (+ <code>buscar_en_la_web</code> , <code>buscar_en_documentos</code> reescrita)

Diagrama de arquitectura: `v5_rag.drawio.xml` .